Admission Control for Probabilistic Callers: Bounding Disclosure When the Query Planner Is a Language Model

Eunice Lee Sophia Shim Caleb Lee
J1729Labs
director@j1729labs.online

Abstract

A gateway that answers aggregate queries over an encrypted store for a language-model agent — a Model Context Protocol server, typically — is a statistical database whose querier has changed character. It is generated from natural language rather than motivated, it can be steered by untrusted content that reaches the model's context, and its queries are cheap enough to issue in bulk. The disclosure controls built for human analysts between 1977 and 1990 therefore transfer in form and fail in force.

We define the adversary that results, enumerate the channels through which a tool's return value — not the tool call — reaches it, and give an exact linear-algebraic criterion for compromise together with three corrections to its naive application. On a synthetic corporate ledger of 5,000 firms, a minimum-query-set-size rule leaves every value-recovery attack at a 1.00 success rate, an individual record falling to two admitted queries. We prove a containment bound of $B \log_2 M$ bits per subject per window, sharpen it by the rank of the admitted queries, show that its record-recovery form is tight to within a factor of two, and prove that a budget keyed to the agent's credential admits no such bound at all — which we confirm by exceeding it. Under taint-gated tool surfaces the injection-attributable share is $B_\tau \log_2 M_\tau$, and we quantify how that degrades when taint tracking leaks. Our claim is containment, not prevention: the admit/deny sequence cannot be closed while the policy remains useful, and no configuration we swept reaches zero attack success at full analyst utility — the best containment costs twenty-five points, and that frontier is the result.

Keywords: admission control; statistical-database inference control; query auditing; tracker attacks; prompt injection; Model Context Protocol; LLM agent security; homomorphic encryption; disclosure bounds; identity propagation.

1 Introduction

A pattern is becoming common in enterprise deployments of language-model agents. Rather than placing sensitive data into the model's context, the data stays in an encrypted store and the model is given *tools*: aggregate, compare, count, search. The model decides what to ask; a gateway decides what to answer. In the Model Context Protocol (MCP) [33] this is expressed as a server that publishes a catalogue of tools and resources; the model's inference loop calls them. The architectural appeal is real. The model never sees the base table, the gateway never has to parse natural language, and the interface between them narrows from an entire prompt to a handful of function signatures.

The appeal conceals an old problem. A gateway that answers SUM, COUNT and AVG over a filtered subset of records *is* a statistical database, and the question of how much an adversary can learn from a sequence of individually innocuous aggregates was studied intensively from the late 1970s. That literature produced

tracker attacks [16, 41], query-set-size control and its defeat [20], overlap control and auditing [10, 11], a textbook treatment of the whole area [17], and a comparative synthesis that reads today as a list of the mistakes a gateway designer is about to make [1].

Our starting observation is that this literature rests on three assumptions about the querier, and that an MCP gateway breaks all three.

The querier is generated, not motivated. Classical analyses assume a human analyst with an information goal, who composes queries deliberately, and much of the classical machinery is calibrated to that picture: the analyst's intent is what an auditor tries to infer, and the analyst's patience is what bounds the attack. Here queries are emitted by a decoding loop from a natural-language instruction. There is no intent to model, and the mapping from instruction to query stream is not stable across models, prompts or even runs. A defence that reasons about why a query was asked has nothing to reason about.

The querier is steerable by content it reads. Indirect prompt injection [24] means that a document, an email, a web page or a meeting transcript that lands in the model's context can determine which tools the model calls and with what arguments. The querier is therefore partly under adversarial control even when the human user is entirely benign, which is a situation the statistical-database literature never had to consider: its adversary was the querier, not a third party acting through one. The practical consequence is that authorisation and intent come apart. The calls carry a legitimate user's authority and an attacker's purpose.

Queries are cheap and bulk. A human analyst mounting a tracker attack was a recognisable event, and the classical bounds — so many queries answerable before compromise — were sized against a person working by hand. A model issues two hundred aggregates in the time it takes to render a paragraph, so the entire classical budget is consumed before anyone could notice it being spent. This inverts the economics of the defences as well as the attacks: any control whose per-query cost grows with history is now on the wrong side of the exchange, and §8.4 measures exactly how badly.

Claim level. We state at the outset what this paper does not claim. It does not claim to prevent prompt injection. We regard a paper that claims to have done so as having destroyed its own credibility in one sentence, and a product making that claim as a liability in regulatory review. What we claim is *containment*: an explicit upper bound on how much an adversary who fully controls the model's tool-calling behaviour can extract, together with an honest account of the channels that remain open. Bounds of this shape survive scrutiny; prevention claims do not.

Threat model and egress. We define $\mathscr{A}_{\mathsf{inject}}$, an adversary who controls part of the model's context and therefore the query stream, and we enumerate the channels through which a tool's return value reaches that adversary (§3). The organising observation is that the tool call is not the disclosure point. An admitted call whose answer never crosses the trust boundary has disclosed nothing, and a refused call can still carry a bit outward, so the object to instrument is the path taken by the return value rather than the call itself.

What transfers from the classical controls, and what does not. Under a filter API that accepts a disjunction of conjunctions — the natural shape for an MCP tool argument — minimum query-set size admits a two-query individual tracker, and the classical four-query general tracker recovers sub-k cell aggregates whenever $n \geq 4k$ (§4). Neither result is new as mathematics; both are new as statements about a gateway whose catalogue publishes exactly the metadata an adversary needs in order to instantiate them.

An exact compromise criterion, and three corrections to it. We give a linear-algebraic criterion that decides compromise exactly, and then correct three ways of applying it wrongly (§5). It must cover sub-k cells and not only individual records. It must cover every value-bearing aggregate, since AVG over a publicly sized set is affine in SUM — an exemption our own gateway shipped with. And the tractable form of the sub-k test is a relaxation rather than a decision procedure, because the exact problem is a minimum-weight-codeword problem.

Bounds, and what they rest on. We prove a containment bound of $B \log_2 M$ bits per subject per window, sharpen it by the rank of the admitted queries (Theorem 20), and show that its record-recovery form is tight to within a factor of two (Theorem 26). We prove that a budget keyed to the agent's credential admits no bound of this form at all, which makes end-user identity propagation a precondition rather than a feature (Theorem 13). We give the injection-attributable refinement under taint gating, quantify how it degrades when taint tracking is imperfect (Theorem 32), and prove that the policy is simulatable, so its refusals disclose nothing about the protected column beyond what the published metadata already gives away.

Evaluation. We evaluate eight attacks — including one that exfiltrates through an external-effect tool and one that transmits a message through admit/deny decisions alone — against ten policy configurations on a synthetic low-cardinality corporate ledger (§8). We report the security/utility frontier including the configurations that fail, and we measure decision latency against the cost of the EC-ElGamal aggregation the gateway performs, which turns out to be the smaller of the two.

Relationship to adjacent work. This paper is the fourth in a series. It assumes an additively and multiplicatively homomorphic elliptic-curve construction (P1) and a formalisation of the system's adversaries and leakage decomposition (P2), and it complements a quantification of re-identification and disclosure budget (P3). It is self-contained: §3 restates what is needed.

We are not aware of prior work connecting classical statistical-database inference control to language-model agents or to MCP. The agent-privacy literature we surveyed reaches for differential privacy and for contextual integrity, and does not engage the pre-2006 disclosure-control line at all. We offer that as the result of a search rather than as an exhaustive claim: a workshop paper or thesis chapter could exist below search visibility, and nothing in this paper depends on the negative holding.

2 Background and related work

2.1 Statistical-database inference control

The field's founding result is that suppressing small query sets is not enough. Denning et al. [16] showed that an *individual tracker* — a predicate that pads an under-sized query set into the admissible range — lets an attacker recover a suppressed statistic from admissible ones, and that a *general tracker* works for every target once found. Denning and Schlörer [15] gave a procedure for finding one quickly; Schlörer [41] quantified how abundant trackers are in real tables. Dobkin et al. [20] studied compromise under a minimum query-set size k, a maximum pairwise overlap r, and ℓ values known in advance, and established a bound of order $(2k-\ell-1)/r$ on the queries that can be answered safely; specialised to $r=1$ and $\ell=0$ this is the familiar $2k-1$. Chin and Özsoyoğlu [10, 11] developed auditing and overlap-based design, and reported its cost. Beck [2] explored perturbation as an alternative. Adam and Worthmann [1] surveyed the whole programme.

Two later results matter to us directly. Kleinberg et al. [29] showed that deciding whether a set of answered queries determines a value is intractable in general, and Kenthapadi et al. [28] showed that an

auditor which denies on the basis of the *data* leaks information through its denials; safety requires the decision to be simulatable from public information. Nabar et al. [34] developed randomised auditing in that frame. Dinur and Nissim [19] gave the reconstruction result that motivated the move to differential privacy [21, 22]. We do not claim differential privacy here, for a reason we make explicit in §6: a gateway that returns exact aggregates cannot.

2.2 Prompt injection and agent defences

Perez and Ribeiro [38] demonstrated direct prompt injection; Greshake et al. [24] established the indirect form, where the payload arrives in retrieved content rather than from the user. Debenedetti et al. [13] built the standard evaluation environment for agent-level attacks and defences. Model-level defences include structured queries [8], preference optimisation [9] and task-specific fine-tuning [39]; Liu et al. [30] give automated attacks. System-level defences are closer to our position: the dual-LLM pattern [45], information-flow control [12, 46], capability and privilege control [14, 42], execution isolation [47], and the design-pattern synthesis of Beurer-Kellner et al. [4]. Industry guidance converges on restricting the combination of untrusted input, sensitive data and external communication [18, 32, 37], and risk-management frameworks ask for exactly the kind of stated residual this paper tries to supply [35].

Our work is complementary rather than competing. Those defences ask which tool calls should be permitted to occur. We ask a question they leave open: given that a sequence of individually permitted, individually innocuous aggregate calls has occurred, how much has been disclosed, and what is the bound?

2.3 MCP security

MCP-specific security work is young. Hou et al. [26] survey the landscape and threat classes; Radosevich and Halloran [40] demonstrate exploits and an automated scanner; Yang et al. [48] benchmark attack surfaces. Tool poisoning — hidden instructions in a tool description the approving user never reads — was described by Beurer-Kellner and Fischer [3], and the specification itself warns that tool descriptions from untrusted servers must be treated as untrusted [33]. The authority-confusion at the centre of the problem is Hardy's confused deputy [25] in modern dress: the gateway acts with its own privileges on instructions that originate elsewhere.

2.4 Encrypted-store leakage

Our gateway sits over an EC-ElGamal [23] store supporting addition and one level of multiplication, in the family of Boneh et al. [5]. The relevant methodological inheritance is from structured encryption's explicit leakage functions [7] and from the leakage-abuse literature that punishes designs which leave leakage implicit [6, 27, 36]. This paper concerns the query-interface leakage; the storage-layer leakage is the subject of the companion threat-model work.

3 System and threat model

3.1 System model

Figure 1 shows the arrangement. Let D be a table of n records. Each record i carries public attributes $A(i)$ — attributes that appear in the tool catalogue and that an adversary may know or guess — and a protected numeric value $x_i \in \mathcal{X}$, stored encrypted. Write $x = (x_1, \ldots, x_n)$.

A *query* is a pair $q = (\varphi, f)$ where φ is a predicate over public attributes with characteristic set $C_\varphi \subseteq [n]$, and $f \in \{\text{COUNT}, \text{SUM}, \text{AVG}, \text{VAR}\}$. The *additive* statistics are COUNT and SUM; these are the ones the

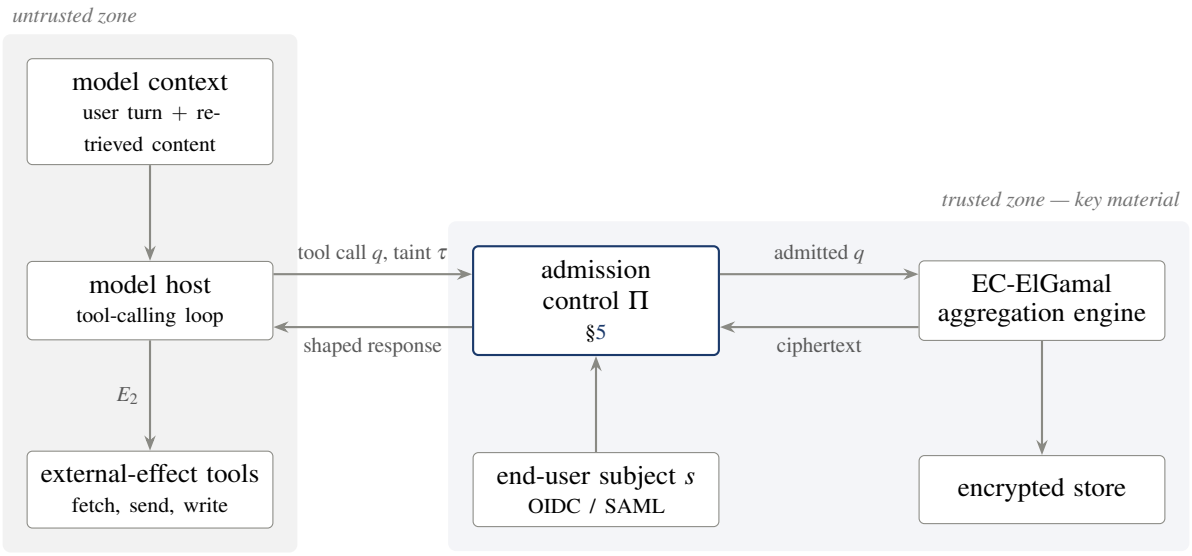


Figure 1: The gateway. Key material never enters the model host. The adversary $\mathscr{A}_{\mathsf{inject}}$ controls part of the model context and therefore the tool-call stream; the disclosure question is not which calls occur but where their return values go.

classical trackers exploit, though §5.2 shows that a criterion restricted to them is no criterion at all. The engine evaluates f over ciphertexts and the gateway returns a *shaped* response $\mathrm{shape}(f(x|_{C_\varphi}))$ drawn from a set of size at most M. When responses are banded, M is the number of bands; when they are exact, M is the size of the response alphabet, which exceeds the per-record domain $|\mathscr{X}|$ — we instantiate it at $|\mathscr{X}|$ only in the restricted sense that Remark 25 sets out.

The gateway also publishes a catalogue: table and column names, sensitivity grades, supported operations, minimum aggregation sizes. This is deliberate — it lets the model plan without seeing a single value — and it is also, as §4 shows, exactly the information an adversary needs to construct a tracker.

3.2 Adversaries

We inherit two adversaries from the companion threat model and add the one this paper is about.

$\mathscr{A}_{\mathsf{host}}$, the model host. It observes prompts, responses, tool-call arguments and return values, and can correlate them across sessions. It holds no key material, which is the structural benefit of keeping the gateway on the far side of the transport: the decryption boundary is never inside the process that runs the model.

$\mathscr{A}_{\mathsf{infra}}$, the infrastructure operator. It observes stored ciphertexts, access patterns and timing, and likewise holds no key material. This is the adversary the encrypted store exists to defeat, and it is the subject of the companion work rather than of this paper.

$\mathscr{A}_{\mathsf{user}}$, the authorised analyst. It holds legitimate tool permissions and attempts to recover individual values by repeated aggregation. This is the classical statistical-database adversary, transplanted unchanged, and it is the one against which the size, overlap and auditing controls of §5 were originally designed.

$\mathscr{A}_{\mathsf{inject}}$**, injected content, which is the subject of this paper.** It is the adversary the classical literature had no occasion to define, because its distinguishing property is that it is neither the querier nor the observer but acts through both. We give it a definition rather than a description, because the bounds in §6 quantify over it.

Definition 1 ($\mathscr{A}_{\mathsf{inject}}$)**.** $\mathscr{A}_{\mathsf{inject}}$ controls a subset of the content that enters the model's context: documents, email bodies, web pages, retrieved passages, meeting transcripts, tool descriptions from third-party servers. It does not control the model weights, the gateway, the key material, or the human user's instruction. Its capability is to influence which tools the model calls, with what arguments, and in what order, and to attempt to have return values routed somewhere it can observe.

Definition 1 is deliberately generous, and we make it more so.

Assumption 2 (Worst-case compliance)**.** We evaluate at compliance $\rho = 1$: every instruction $\mathscr{A}_{\mathsf{inject}}$ places in context is followed by the model. All bounds in §6 are stated for an adversary that chooses the admissible query stream directly.

This assumption is what makes the results model-independent. Our defence operates below the model, on the tool-call stream; it never inspects natural language. Whether a particular model complies with a particular payload is an empirical question about that model, and a moving one. By taking $\rho = 1$ we remove it from the analysis, at the cost of reporting a conservative bound. §8.7 verifies that measured leakage under our policy is flat in ρ while leakage under size control alone is linear in it.

3.3 Egress channels

The disclosure point is not the tool call. A gateway that logs every call and alerts on suspicious ones has instrumented the wrong event: an admitted call whose return value never leaves the trust boundary has disclosed nothing to $\mathscr{A}_{\mathsf{inject}}$, and a denied call can still carry a bit outward. What matters is the path from the return value to the adversary. We distinguish three.

E_1**, direct echo.** The value appears in the assistant's visible output. Capacity is high — an answer is exactly what the user asked for, so nothing constrains its size — but the output is delivered to the legitimate user. The channel is therefore useful to $\mathscr{A}_{\mathsf{inject}}$ only when the user is expected to forward, publish or paste the response, or when the attacker is also the user. It is the channel a deployment controls by controlling who reads the session, rather than by any gateway rule, and we exclude it from the injection-attributable accounting for that reason.

E_2**, argument encoding.** The value is encoded into the arguments of a subsequent tool call with external effect: a URL fetched, a message sent, a file written, a webhook invoked. Capacity is essentially the argument length, hundreds of bits per call, and delivery is direct and immediate. This is the channel that matters. It is also the one that the deployment, rather than any single component, creates: connecting an aggregation server and a fetch server to the same model composes two correct pieces into a confused deputy. Taint gating closes it, and §8.2 measures that closure rather than assuming it.

E_3**, the decision and format side channel.** The adversary learns whether a call was admitted, how long it took, or what shape the response took. Capacity is low — we treat it as one bit per call — but unlike the other two it cannot be closed by any policy worth deploying, because a policy that never denies is not a policy. Proposition 34 states that precisely, and Proposition 33 explains why the residual is nonetheless tolerable: the bits it carries are not bits about the protected column.

Remark 3. E_2 is where MCP's composability becomes a liability: a deployment that connects an aggregation server and a fetch server to the same model has, without any single component being at fault, created a confused deputy in the sense of Hardy [25]. This is the mechanism behind the industry guidance to avoid combining untrusted input, sensitive data and external communication [32]; our contribution is to make the residual quantitative rather than advisory.

3.4 Non-goals

We do not defend against: a human user with full legitimate authority who chooses to exfiltrate data; content pasted by the user directly into the prompt, which never passes through the gateway; an actively malicious model host that fabricates subject identities; compromise of the key material; or physical and microarchitectural side channels. We do not prevent injection. We do not achieve differential privacy in the exact-response mode, and §6 says why.

4 What the classical controls do and do not buy

Throughout this section the policy enforces only the classical size rule: admit q iff $k \le |C_\varphi| \le n-k$. We assume the filter language of the tool API accepts a disjunction of conjunctions over public attributes, which is the natural JSON shape for an MCP tool argument and the shape our reference implementation uses.

Proposition 4 (Two-query individual tracker)**.** *Let record t be uniquely characterised by public attributes, i.e. there is a conjunction ψ_t with $C_{\psi_t} = \{t\}$. Suppose there exists an admissible predicate θ with $t \notin C_\theta$ and $k \le |C_\theta| \le n-k-1$. Then two admitted* SUM *queries reveal x_t exactly:*

$$x_t \;=\; q(\theta \vee \psi_t) \;-\; q(\theta).$$

Proof. $C_{\theta \vee \psi_t} = C_\theta \cup \{t\}$ has size $|C_\theta| + 1 \in [k+1, n-k]$, so both queries are admitted. SUM is additive, so the difference is x_t. □ □

Proposition 4 is not deep, and that is the point. It costs two calls, it needs no search, and the catalogue the gateway publishes in order to be useful tells the adversary which θ to pick. In our evaluation (§8.2) it recovers 100% of targeted records at exactly 2.0 admitted queries per record.

If the filter language is restricted to a single conjunction, the disjunction in Proposition 4 is unavailable, but differencing is not: a pair of nested conjunctions whose sizes differ by one serves the same purpose. Our population contains 177 such pairs, and the attack needs one primitive beyond conjunction — a $\neq$ or exclusion filter — which every practical filter API provides. Restricting the filter language is therefore not a defence; it is an inconvenience.

Proposition 5 (General tracker; 16)**.** *Let $n \ge 4k$ and let T be a predicate with $2k \le |C_T| \le n-2k$. For any predicate φ with $|C_\varphi| < k$ and any additive f,*

$$q(\varphi) \;=\; q(\varphi \vee T) \;+\; q(\varphi \vee \neg T) \;-\; q(T) \;-\; q(\neg T),$$

and all four query sets have size in $[k, n-k]$.

Proof. Additivity gives $q(\varphi \vee T) = q(\varphi) + q(T) - q(\varphi \wedge T)$ and likewise for $\neg T$; summing and using $q(\varphi \wedge T) + q(\varphi \wedge \neg T) = q(\varphi)$ yields the identity. For the sizes: $|C_{\varphi \vee T}| \ge |C_T| \ge 2k \ge k$ and $|C_{\varphi \vee T}| \le |C_\varphi| + |C_T| \le (k-1) + (n-2k) = n-k-1$; symmetrically for $\neg T$, whose size lies in $[2k, n-2k]$ by construction. □ □

Proposition 5 recovers the aggregate of a cell that is *below* the minimum query-set size — precisely the population the size rule exists to protect. An adversary that reuses one padding set pays two admitted queries per cell, since $q(T)$ and $q(\neg T)$ amortise; our implementation re-issues them per cell so that per-record exposure does not concentrate, and therefore measures 4.0 admitted queries per recovered cell. The condition $n \geq 4k$ is necessary but not sufficient, since a suitable T must also be *expressible* in the filter language; at $n = 5{,}000$ the single public attributes alone supply one for every k a deployment would tolerate.

Why low cardinality makes this worse here. Our setting is corporate rather than personal data — legal entities, not individuals — and this cuts against the defender. In the population of §8.1, 77.4% of occupied cross-classification cells hold fewer than $k = 5$ records and 9.8% of records are uniquely identified by four public attributes alone. The premise that a query set of size $\geq k$ is a crowd is simply false at these cardinalities.

Proposition 6 (Query economics)**.** *The classical bound of Dobkin et al. [20] limits safe answering to order* $(2k - \ell - 1)/r$ *queries, a quantity calibrated to an analyst issuing queries by hand. In the agent setting the entire budget is consumed in milliseconds. In our harness, recovering* 100 *records by Proposition 4 takes* 200 *admitted queries and, at the* 0.017 *ms median decision cost of a pure size rule, under* 10 *ms of gateway decision time in total.*

The consequence is architectural, and it is the reason §5 treats cost as a first-class design constraint rather than an implementation detail. A control whose per-query cost grows with history — which is what overlap control and auditing are — has to be paid for at machine query rates. We measure this in §8.4: hard pairwise overlap checking grows 28.7× over 1,500 queries.

5 Admission control design

The policy Π is a composition of five rules. We name them D_1–D_5 and treat them as a ladder, because the evaluation shows each one failing in a specific and instructive way when used alone.

5.1 D_1: minimum query-set size

Admit iff $k \leq |C_\varphi| \leq n - k$. Both tails matter: a complement query recovers everything the direct query would. Propositions 4 and 5 say this is necessary and badly insufficient.

We keep it, and the sweep of §8.5 says why: it is free (0.017 ms at the median, flat in history), it sheds load before the expensive rules run, and it is not redundant with auditing. Holding the cell-aware auditor fixed and varying k alone, attacks still succeed at 0.82 at $k = 0$ and 0.55 at $k = 3$, and close completely at $k = 5$; the size rule is what shuts them, at a cost of twenty-two points of benign utility. Raising k further buys nothing and costs another twenty and thirty-seven points. The oldest control in the literature is neither sufficient on its own nor redundant once auditing is present.

5.2 D_2: differencing control

Differencing is a linear-algebra phenomenon and is best treated as one. Let $Q \in \{0,1\}^{m \times n}$ collect the characteristic vectors of the admitted additive queries, with answers $b = Qx$.

Theorem 7 (Exact compromise criterion)**.** *Work over* $\mathbb{Q}$ *and suppose the adversary's only constraint on* x *is* $Qx = b$*. For* $S \subseteq [n]$*, the aggregate* $\sum_{i \in S} x_i$ *is determined by* b *if and only if* $\mathbf{1}_S \in \mathrm{rowspace}(Q)$*. In particular* x_i *is determined iff* $e_i \in \mathrm{rowspace}(Q)$*.*

Proof. If $\mathbf{1}_S = \lambda^\top Q$ then $\sum_{i\in S} x_i = \lambda^\top b$, which is a function of b. Conversely the solution set is the affine space $x_0 + \ker Q$. If $\mathbf{1}_S \notin \text{rowspace}(Q) = (\ker Q)^\perp$ then there is $v \in \ker Q$ with $\mathbf{1}_S^\top v \neq 0$, and $x_0 + \varepsilon v$ is consistent with b for all ε while its S-aggregate varies. □ □

Remark 8 (Bounded domains)**.** Over a bounded integer domain $\mathscr{X}$ the converse requires that $x_0 \pm \varepsilon v$ remain in $\mathscr{X}^n$ for some $\varepsilon \neq 0$. The criterion is therefore exact for unbounded or sufficiently wide domains and conservative otherwise — it can declare safe a configuration that a domain-aware attacker can still pin down. Deployments with narrow value domains should treat it as necessary, not sufficient.

Theorem 7 yields a decidable auditor. Maintain the reduced row echelon form (RREF) of Q over a prime field; admit q only if the resulting span contains no standard basis vector.

Lemma 9 (Weight-one test is exact)**.** *Let R be the RREF of Q. Then $e_i \in \text{rowspace}(Q)$ if and only if R contains a row of Hamming weight one, namely e_i itself.*

Proof. ($\Leftarrow$) Immediate. ($\Rightarrow$) If $e_i \in \text{rowspace}(Q)$ its leading nonzero is in column i, so i is a pivot column; let r be the row with pivot i. Writing $e_i = \sum_j c_j r_j$ and reading off pivot columns gives $c_j = (e_i)_{p_j}$, which is 1 for $p_j = i$ and 0 otherwise; hence $e_i = r$. □ □

First correction: the criterion must cover sub-k cells. Lemma 9 covers compromise of an *individual* value. It does not cover the disclosure that Proposition 5 actually achieves, which is the aggregate of a cell of size $2 \le |C| < k$ — a k-anonymity violation in the sense of Sweeney [43] even though no single record is pinned down, and the attribute-diversity refinements that followed k-anonymity [31] do not help, because the disclosure is of an aggregate rather than of a tuple. In our evaluation the classical auditor stops every individual-compromise attack and still leaves the general tracker at a 0.77 success rate; the 23% it does catch are exactly the singleton cells, for which the weight-one test fires. We therefore define the *cell-aware* test: refuse when the resulting RREF contains a row of Hamming weight in $[1, k-1]$.

Second correction: the criterion must cover every value-bearing aggregate. Theorem 7 is stated for additive statistics, and it is tempting to audit only those. That is a mistake, and it was ours: an earlier version of this gateway audited SUM and exempted AVG. Because $|C_\varphi|$ is computable from public attributes — the gateway publishes the catalogue that makes it so — the two are affine in each other, and $x_t = \text{avg}(\theta \vee \psi_t)\,(m{+}1) - \text{avg}(\theta)\,m$ recovers a record from two AVG calls the auditor never saw. Against the exempting version this recovered 100 of 100 targeted records under the cell-aware policy and, as $\mathscr{A}_{\mathsf{user}}$, 32 of 100 under the full recommended stack, where only the budget limited it. The rule is therefore: every aggregate that is a function of the protected values is audited on its characteristic vector, in a basis indexed by the column and the moment it reveals. SUM and AVG share the first-moment basis; VAR gets a second-moment basis of its own; COUNT reveals only a set size, which is public, and is exempt. We report this because the exemption is natural, the reasoning behind it is plausible, and the resulting hole is total.

Remark 10 (Honest status of the cell-aware test)**.** The weight-one test is exact (Lemma 9); the cell-aware test is not. Deciding whether *any* $0/1$ vector of weight below k lies in a given subspace is a minimum-weight-codeword problem, NP-hard in general [44], which is the concrete form Kleinberg et al.'s intractability result takes here [29]. The RREF-row test is a tractable relaxation that errs in both directions: a low-weight RREF row need not correspond to a realisable cell (over-refusal), and a low-weight $0/1$ combination need not appear as an RREF row (under-refusal). We report its measured behaviour rather than claiming soundness: it reduced general-tracker success from 0.77 to 0.00 at no measurable cost on our benign workload, which it admits at exactly the same rate as the classical auditor (572 of 726).

Remark 11 (The shipped auditor is not the mathematical one)**.** Theorem 7 and Lemma 9 are stated over $\mathbb{Q}$; the implementation works over $\mathrm{GF}(2^{31}-1)$ so that products fit a machine word, and the substitution is not free in the direction one would hope. A vector independent over $\mathbb{Q}$ can be dependent modulo p, in which case the auditor treats it as carrying nothing and admits it without recording it — an under-refusal, not an over-refusal. The two ranks agree unless p divides some $\mathrm{rk} \times \mathrm{rk}$ minor of the admitted matrix, and Hadamard's bound puts those minors below $\mathrm{rk}^{\mathrm{rk}/2}$, which passes 2^{31} by $\mathrm{rk} = 16$ and is astronomically larger at the ranks our sessions reach. We therefore have no guarantee, only the observation that no such collision occurred in our runs. A deployment that wants the criterion rather than an approximation to it should use rational or multi-prime arithmetic and pay for it.

A cheap heuristic, and its failure. Because auditing is expensive (§8.4), we also evaluate a constant-cost alternative: maintain a per-record *exposure* counter e_i = number of admitted query sets containing i, and refuse when $\max_{i \in C_\varphi} e_i \geq E$. It costs $O(|C_\varphi|)$ and is flat in history. It is also, we find, a poor defence, and we report it as a negative result: it blocks attacks only to the extent that they reuse a base predicate, and differencing across a genuinely disjoint family is untouched. Our A2 does reuse base predicates and so survives at 0.71; a disjoint family survives at 1.00. Either way the cap costs 20.7% of benign queries. Two exposures per record is all differencing needs.

The failure is structural rather than a matter of tuning, which is worth stating precisely because the heuristic is otherwise so attractive.

Proposition 12 (Per-record exposure capping fails for every $E \geq 2$)**.** *Let $(\alpha_j, \alpha_j \wedge \beta_j)_{j \leq J}$ be nested pairs of predicates over public attributes with $|C_{\alpha_j \wedge \beta_j}| = 1$, $k+1 \leq |C_{\alpha_j}| \leq n-k$, and $C_{\alpha_1}, \ldots, C_{\alpha_J}$ pairwise disjoint. Under a size rule together with an exposure cap $E \geq 2$, every one of the $2J$ queries $q(\alpha_j)$, $q(\alpha_j \wedge \neg\beta_j)$ is admitted, and the adversary recovers J individual values. Under $E = 1$ no two admitted query sets may intersect, so the admitted family is a partition of a subset of $[n]$.*

Proof. Disjointness means a record's exposure is affected only by the pair it belongs to. Before the pair is issued every such record has exposure 0; after the first query it has exposure $1 < 2 \leq E$, so the second query is admitted as well. Differencing the two admitted answers gives the value of the single record in $C_{\alpha_j \wedge \beta_j}$. For $E = 1$, a record in the intersection of two admitted sets would have exposure 1 when the second is tested, which the rule refuses. □ □

Two exposures per record is precisely what differencing requires, so the only setting that defeats it is the one that forbids any two admitted query sets from meeting. That confines the admitted family to a partition, to which Proposition 28 then applies — though at our parameters the ceiling it yields sits far above the budget, so what collapses the analyst's workload at $E = 1$ is the refusals themselves rather than any rank limit.

The measured 0.71 is in fact weaker than the proposition predicts, and the gap is instructive. Our A2 implementation reuses one base predicate for every singleton size band inside it, so its 177 targets occupy only 125 distinct base sets; under $E = 2$ exactly the first target in each base set succeeds, giving $125/177 = 0.71$. On a genuinely disjoint family, Proposition 12 gives recovery of every target; we did not implement that variant, so the 1.00 is a consequence of the proposition rather than a measurement. The exposure cap therefore does partial work against an adversary careless enough to reuse a base predicate and none at all against one that does not, which strengthens the negative result rather than softening it. Tightening to $E = 1$ stops both variants, at a benign task utility of 0.30 against 0.75 for the cell-aware auditor at the same k.

5.3 D_3: budget scope

Let the policy admit at most B queries per accounting principal per time window. The question is what the principal is. The natural implementation — the one an MCP server gets for free — keys the budget to the credential presented by the agent. This is worthless.

Theorem 13 (Credential-scoped budgets admit no containment bound)**.** *Let Π_{cred} grant budget B per credential, and suppose the adversary can cause new credentials to be issued at negligible cost — as it can whenever opening a new agent session yields one. Then for every B and every target set S there is an adversary recovering $\{x_i\}_{i\in S}$ using $\lceil |S|/\lfloor B/2\rfloor \rceil$ credentials. Consequently leakage is not bounded by any function of B and M, and no bound of the form of Theorem 16 holds. Under Π_{subj}, which keys the budget to the authenticated end-user subject s, the number of admitted queries attributable to s in a window is at most $B(s)$ regardless of how many credentials are used.*

Proof. By Proposition 4 each target costs two admitted queries; partition S into blocks of $\lfloor B/2\rfloor$ and present a fresh credential per block. Each block is admitted in full because its budget is fresh, and $|S|$ may be taken as large as the table, so no function of B and M bounds the result. (Leakage is of course still at most $H(x)$; the point is that the containment bound has no content.) For the second claim, the accounting key is s, which by hypothesis is authenticated upstream and carried on every call, so all blocks debit the same counter. □ □

Corollary 14 (Identity propagation is a precondition)**.** *Every containment bound in §6 presupposes that each tool call carries an authenticated end-user subject. A gateway that receives only the agent's API key cannot state any of them.*

We stress the direction of this result. Identity propagation is usually motivated as a product requirement — a call-centre agent and a branch manager should not receive the same answer. Corollary 14 shows it is prior to that: without it the security argument has no subject to quantify over. It is also, in our measurements, the difference between a rotation attack at 1.00 and the same attack at 0.32, and — run against 400 targets rather than 100 — the difference between 8,041 bits recovered against a 1,287-bit bound and 643 bits against the same bound (§8.6). Retrofitting it is a redesign, because the budget, the audit state and the taint state are all keyed by it.

5.4 D_4: taint-gated tool surface

This is the rule that has no classical antecedent, because the classical querier could not be steered. Let $\tau \in \{0,1\}$ indicate whether the current model context contains content from an untrusted source. When $\tau = 1$ the gateway reduces its own surface along four axes at once. The admissible aggregate set shrinks, in our configuration to COUNT alone. Responses are banded rather than exact, which reduces the alphabet from M to M_τ and is what gives Theorem 30 its content. A separate and much smaller budget B_τ applies to admitted queries, while a separate cap C_τ applies to calls, admitted or not. And tools with external effect are refused outright, which closes E_2.

The distinction between B_τ and C_τ is not pedantry. A budget on admissions does not bound the decision channel E_3, because a refused call still returns its refusal; only a cap on calls does. We introduce C_τ for that reason and return to it in Proposition 34. The refusal of external-effect tools is implemented and measured rather than assumed: our harness carries an `export` tool, and the attack of §8.2 counts a value as leaked only when it both is recovered and crosses that tool.

Taint is supplied by the host, not inferred by the gateway, and its soundness is an assumption we state rather than prove.

Assumption 15 (Taint soundness)**.** Every tool call issued while adversary-controlled content is present in the model's context is presented to the gateway with $\tau = 1$.

Assumption 15 is the load-bearing assumption of Theorem 30, and it is the one most likely to fail in deployment: content laundered through a summarisation step, a sub-agent, or a cached retrieval may arrive untagged. We discuss failure modes in §9. Existing information-flow machinery for agents [12, 14, 46] is the natural supplier of this bit, and our design is deliberately parameterised on it rather than reimplementing it.

5.5 D_5: deny-by-default on unclassified columns

Columns absent from the policy catalogue are refused. This looks like hygiene and is in fact a completeness condition for the bound: Theorem 16 quantifies over responses about classified columns, so an unclassified column is outside the accounting entirely and the bound says nothing about it. Measured, the difference is a 1.00 success rate against a shadow column versus 0.00. The practical difficulty is not the rule but the classification: a gateway that requires the customer to enumerate their own sensitive columns is a consulting engagement, not a product. Automated schema classification at provisioning time is the intended supplier, and its recall is a direct input to this rule's effectiveness.

6 Containment bounds

6.1 The basic bound

Theorem 16 (Containment)**.** *Let Π admit at most B queries per subject per window and let every admitted response take values in a set of size at most M. Let V be any function of the admitted responses and of public information. Then*

$$I(x; V \mid \text{pub}) \;\le\; B \log_2 M \quad \textit{bits.}$$

If in addition the adversary can export at most κ bits through the channels of §3.3 during the window, then $I(x; V_{\mathscr{A}}) \le \min(B \log_2 M,\ \kappa)$.

Proof. The admitted responses form a vector R taking at most M^B values, so $H(R \mid \text{pub}) \le B \log_2 M$. V is a (possibly randomised) function of R and public information, so by the data-processing inequality $I(x; V \mid \text{pub}) \le I(x; R \mid \text{pub}) \le H(R \mid \text{pub})$. For the second claim, $V_{\mathscr{A}}$ is a function of the at most κ bits that cross the boundary; apply data processing again. □ □

Remark 17 (No budget, no bound)**.** The theorem is conditioned on a budget. A policy with no budget — which includes every configuration in our ladder up to and including the exposure cap — has no containment bound at all, and we report it that way rather than substituting some finite B chosen after the fact. This is the sense in which D_3 is not one control among five but the one that makes a bound sayable.

Corollary 18 (Records recoverable)**.** *If the x_i are independent and uniform on $\mathscr{X}$ and the adversary learns x_i exactly for all $i \in S$, then $|S| \le B \log_2 M / \log_2 |\mathscr{X}|$.*

Remark 19 (Exact responses make the bound vacuous)**.** Theorem 16 has content only when M is small. A gateway that returns exact aggregates has $M \ge |\mathscr{X}|$ — strictly larger, in fact, by Remark 25 — and the record-recovery bound of Corollary 23 degenerates to $|S| \le B$: the bound then says only that you cannot learn more records than you asked questions. This is the same wall that separates exact-aggregate release from differential privacy [19, 22]: exact answers admit no non-trivial information-theoretic guarantee. A deployment must therefore choose, per mode, between numerically useful answers and a tight bound. Our

evaluation quantifies the price of that choice (§8.5); it is not small, and we think stating it plainly is more useful than obscuring it.

The bound of Theorem 16 counts admitted queries. It can be sharpened to count only the ones that carried new information. The accounting has to respect the audit slots of §5.2, because two queries over the same record set can be independent when they concern different columns or different moments; we state the clean case and then say what the others cost.

Theorem 20 (Rank refinement, first-moment aggregates)**.** *Fix a subject, a window and a column c; let the responses be exact, let V be as in Theorem 16, and suppose the admitted aggregates on c are drawn from* $\{\text{COUNT}, \text{SUM}, \text{AVG}\}$. *Let Q_c collect the characteristic vectors of the admitted* SUM *and* AVG *queries on c and put* $\text{rk} = \text{rank}_{\mathbb{Q}} Q_c$. *Then*

$$I(x; V \mid \text{pub}) \;\le\; \text{rk} \log_2 M \;\le\; \min(B, n) \log_2 M.$$

Proof. Choose a maximal linearly independent subset of rk rows of Q_c and let R_0 be the corresponding admitted responses. Each such response is in bijection with the value of its linear functional $\langle v, x \rangle$: for SUM it is that value, and for AVG it is that value divided by $|C_\varphi|$, which is computable from the public attributes and so is available to the conditioning. Every remaining admitted response is the functional value of a rational combination of the chosen rows, hence — with exact arithmetic — a deterministic function of R_0 and the public set sizes. Therefore $H(R \mid \text{pub}) \le H(R_0 \mid \text{pub}) \le \text{rk} \log_2 M$, since R_0 consists of rk genuine responses each drawn from an alphabet of size at most M. Data processing along $x \to R \to V$ gives the claim, and $\text{rk} \le \min(B, n)$. □ □

Remark 21 (Why VAR is excluded)**.** The proof turns on each chosen response being in bijection with the value of a single linear functional, which is what makes its alphabet M. A VAR response is not: it is a function of a first-moment and a second-moment functional of the same query set, and it determines neither. Accounting for it requires one slot per moment order and gives the weaker $I \le \sum_d \text{rk}_d \log_2 M_d$, where M_d bounds the alphabet of a d-th moment functional and is of order $n|\mathscr{X}|^d$ rather than M. We also note that the shipped auditor registers VAR in the second-moment slot alone (Appendix B) and so never records the first-moment functional that this accounting charges for — an under-refusal of the same family as Remark 11, and a defect we did not discover until the bound was written down.

A *refusing* rank cap — one that declines any query raising rk above $\text{rk}_{\max}$ — would therefore bound disclosure by $\text{rk}_{\max} \log_2 M$ however large B is. We flag that this is not the cap our artifact implements. The cap in Appendix B is an availability valve that admits past the limit without recording, which provides no containment at all; making it refuse instead is a one-line change whose cost we have not measured.

Remark 22. The argument needs exact arithmetic and therefore does not survive banding: a banded response is not an exact linear functional of the others, so dependent queries can still carry information about where inside its band each answer fell. Banding compensates by shrinking M instead. The two refinements apply in different modes and should not be multiplied together.

Theorem 20 has a corollary that is what our evaluation actually measures, and stating it separately avoids an equivocation that would otherwise sit under the numbers in §8.6.

Corollary 23 (Record-recovery bound)**.** *Assume exact responses, a value domain wide enough for the converse of Theorem 7 to hold, and x_i independent and uniform on $\mathscr{X}$. Then the number of records whose value the adversary determines exactly in a window is at most* $\text{rk}_1 \le \min(B, n)$, *where* rk_1 *is the first-moment rank of the column in question, and the recovered-value content is at most* $B \log_2 |\mathscr{X}|$ *bits, in the sense of Corollary 18.*

Proof. By Theorem 7, x_i is determined exactly if and only if $e_i \in \text{rowspace}(Q_c)$. Distinct standard basis vectors are linearly independent, so at most $\dim \text{rowspace}(Q_c) = \text{rk}_1$ of them lie in that space, and $\text{rk}_1 \leq \min(B, n)$. Each recovered value contributes $\log_2 |\mathscr{X}|$ bits by uniformity. □ □

Remark 24. Two hypotheses are load-bearing and neither is innocuous in our own evaluation. The bounded-domain caveat after Theorem 7 applies here too, so the corollary bounds what a domain-oblivious adversary recovers. And the population's revenue is log-normal rather than uniform, so $\log_2 |\mathscr{X}| \approx 20.1$ bits per record overstates the true per-record entropy; we use the same convention on both sides of every comparison, which keeps the comparison internally consistent while making the absolute bit figures generous to the adversary.

Remark 25 (Which M is which)**.** In banded mode the response alphabet is the band set and M is simply its size. In exact mode it is not: a SUM over m records ranges over roughly $m|\mathscr{X}|$ values, so the response alphabet is strictly larger than the per-record domain and the general bound of Theorem 16 is correspondingly weaker than $B \log_2 |\mathscr{X}|$. When we instantiate exact mode with $M = |\mathscr{X}|$ — as the evaluation does — the quantity being bounded is the recovered-value content of Corollary 23, not the adversary's total information about x, which can include partial knowledge of many records that no bound of this form captures. We keep the two apart, and report only the former.

The upper bound would be uninteresting if no adversary approached it. One does, and it is the adversary of §4.

Theorem 26 (The record-recovery bound is tight to within a factor of two)**.** *Let the policy enforce a size rule and a budget of B admitted queries with exact responses, let the filter language admit a disjunction of conjunctions, and suppose the public attributes isolate at least $\lfloor B/2 \rfloor$ records, each of which admits a padding predicate θ with $t \notin C_\theta$ and $k \leq |C_\theta| \leq n - k - 1$. If the x_i are independent and uniform on $\mathscr{X}$ and the domain is wide enough for the converse of Theorem 7 to hold, there is an adversary $\mathscr{A}$ recovering $\lfloor B/2 \rfloor$ record values exactly, so that*

$$\lfloor B/2 \rfloor \log_2 |\mathscr{X}| \;\leq\; \sup_{\mathscr{A}} \text{rec}(\mathscr{A}) \;\leq\; B \log_2 |\mathscr{X}|,$$

where rec *denotes the recovered-value content bounded by Corollary 23.*

Proof. Apply Proposition 4 to $\lfloor B/2 \rfloor$ distinct isolated targets S, at two admitted queries each; every query is admitted because each padded set has size in $[k+1, n-k]$. Each x_i, $i \in S$, is then a deterministic function of the view, so $H(x_S \mid V_{\mathscr{A}}) = 0$ and the recovered content is $H(x_S) = |S| \log_2 |\mathscr{X}|$ by independence. The upper bound is Corollary 23. □ □

Remark 27. We state tightness for the recovered-value content rather than for $I(x; V_{\mathscr{A}})$ deliberately. By Remark 25 the exact-mode response alphabet exceeds $|\mathscr{X}|$, so $B \log_2 |\mathscr{X}|$ is not an upper bound on total mutual information — a single exact SUM over m records already carries about $\log_2 |\mathscr{X}| + \frac{1}{2} \log_2 m$ bits. Sandwiching the two quantities together would be exactly the equivocation Remark 25 warns against. Under $x \perp \text{pub}$ the conditioning in Theorem 16 may be dropped, which we do silently from here on.

This is not an asymptotic statement about a contrived adversary: it is what the evaluation measures. Under the size-only policy our tracker recovers 100 records from 200 admitted queries (§8.2) — the construction above with $\lfloor B/2 \rfloor = 100$, which is what a budget of $B = 200$ would have permitted, that policy having no budget at all. Under the subject-scoped policy, where $B = 64$ really is enforced, the adversary recovers 32 records for 643.3 bits, which is $\lfloor B/2 \rfloor \log_2 |\mathscr{X}|$ to the digit: the lower bound of Theorem 26 attained exactly.

Theorem 20 also bounds a structured family of policies in a way that is worth isolating, because it is the family in which the security/utility trade behaves best — and, as we will see, not the family our gateway belongs to.

Proposition 28 (Ceiling for partition-closed policies)**.** *Call a policy* partition-closed *at granularity k if every admitted query set is a union of blocks of a fixed partition of $[n]$ whose blocks all have size at least k. Assume exact responses, first-moment aggregates, and a value domain wide enough for the converse of Theorem 7 to hold. Then* $\mathrm{rk} \le \lfloor n/k \rfloor$*, and by Theorem 20 no party learns more than $\lfloor n/k \rfloor \log_2 M$ bits in a window, whatever budget it is granted. Moreover, for $k \ge 2$ no standard basis vector lies in the span of the block indicators, so by Theorem 7 no individual record is recoverable at all.*

Proof. A partition of $[n]$ into blocks of size at least k has at most $\lfloor n/k \rfloor$ blocks, and every admitted characteristic vector lies in the span of the block indicators, so rk is at most the number of blocks. A nonzero vector in that span is constant on each block, so it cannot have Hamming weight one unless some block is a singleton. □ □

The second half is the interesting one, and it says the trade in this family is *asymmetric in the analyst's favour*: the ceiling constrains both parties equally, but within it the analyst still collects $\lfloor n/k \rfloor$ usable block aggregates while the adversary recovers no records whatsoever. The honest form of the design lesson is therefore not that safety and utility are interchangeable but that a policy which fixes the answerable question space in advance gets a far better exchange rate than one which does not — and our gateway does not, because it accepts arbitrary disjunctions of conjunctions and so is not partition-closed. Generalisation hierarchies, which are partition-closed by construction, are the obvious route to that better exchange rate, and we did not take it.

We are careful not to claim more. Proposition 28 does *not* explain the frontier of §8.5: the swept policies are not partition-closed, and at our parameters the ceiling it yields, $\lfloor 5000/5 \rfloor = 1000$, is fifteen times looser than the budget $B = 64$ that actually binds. The frontier's shape comes from denial counts. In the swept configurations at $k = 5$ every refusal is a genuinely sub-k benign query — 105 of the sweep's 488, with no collateral denials at all — and none comes from a rank ceiling. The 112 taint-surface refusals of Table 4 belong to the P0–P8 ladder, over which the sweep does not vary D_4, and cannot shape this frontier at all.

Finally, the window in Theorem 16 is an accounting device, and it is worth being explicit that renewing it renews the allowance.

Proposition 29 (Composition across windows)**.** *Over W consecutive windows for the same subject, $I(x;V) \le WB\log_2 M$. Under the hypotheses of Theorem 26 the recovered-value content additionally satisfies $W\lfloor B/2 \rfloor \log_2 |\mathcal{X}| \le \sup_{\mathcal{A}} \mathrm{rec}(\mathcal{A}) \le WB\log_2 |\mathcal{X}|$, so long as $W\lfloor B/2 \rfloor$ isolated targets remain.*

Proof. Let R_w be the responses admitted in window w. Then $I(x;V) \le H(R_1,\dots,R_W) \le \sum_w H(R_w) \le WB\log_2 M$ by subadditivity, and the lower bound follows by running the construction of Theorem 26 on fresh targets in each window. □ □

A per-window budget therefore bounds a burst, not a lifetime. A deployment that needs the latter must either decline to renew — a non-replenishing allowance per subject per engagement — or carry state across windows. Our auditor does carry state, since its basis is never reset within a subject; our budget does not. We regard that asymmetry as a defect of the present implementation rather than a design position, and note that it is the one place where the artifact is weaker than the theory it accompanies.

6.2 Injection-attributable leakage

The bound that matters commercially is not the total but the part attributable to injection, because that is the part the customer did not authorise.

Theorem 30 (Injection-attributable containment)**.** *Under Assumption 15 and taint gating with tainted budget B_τ and tainted response alphabet M_τ,*

$$I\big(x; V_{\mathscr{A}_{\mathsf{inject}}}\big) \;\le\; B_\tau \log_2 M_\tau \quad \textit{bits.}$$

Proof. By Assumption 15 every query influenced by $\mathscr{A}_{\mathsf{inject}}$ is issued with $\tau = 1$, so at most B_τ such queries are admitted, each returning one of at most M_τ values. Apply Theorem 16 with the tainted parameters. □ □

At our parameters $B_\tau = 8$ and $M_\tau = 24$, so injection-attributable leakage is at most $8 \log_2 24 \approx 37$ bits per subject per window, against $\log_2 |\mathscr{X}| \approx 20.1$ bits for one firm's revenue: under two records' worth. This is the number we would put in front of a regulator, and it is falsifiable in a specific place — it fails exactly when Assumption 15 fails.

It is also loose by construction under the stack we recommend, and we would rather say so than let the bound do more work than it has earned. P7's tainted surface admits only COUNT, whose answers are set sizes computable from the published catalogue, so the actual injection-attributable leakage about x there is zero rather than thirty-seven bits. The thirty-seven is what the bound permits a deployment that widens the tainted surface, and it is the number that stays true if one does.

Remark 31 (What we do not get to claim)**.** An earlier version of this theorem carried a second term, $\kappa_\tau \le B_\tau$, on the grounds that the decision channel yields one bit per call and at most B_τ calls are made. That is wrong, and instructively so: B_τ caps *admissions*, and a refused call returns its refusal just as informatively as an admitted one returns its answer. Bounding E_3 in general requires the separate call cap C_τ of §5.4, giving $\kappa_\tau \le C_\tau$. Our measured channel happens to close earlier, at B_τ, because the particular encoder we build needs admissions to signal with: once the admission budget is spent, every probe is refused and the decoder sees a constant. That is a property of that encoder and not a bound — one signalling through *which* rule refuses, rather than through whether any does, would survive an exhausted admission budget — so C_τ is the cap we would rely on. Since E_3 carries nothing about x given the public metadata (Proposition 33), the corrected theorem does not need it; the cap matters for exfiltration of context-resident secrets, which is a different asset.

Assumption 15 is the strongest hypothesis in this paper, so it is worth knowing what a violation costs rather than merely that one is fatal. It is not fatal; it is linear.

Theorem 32 (Degradation under leaky taint)**.** *Suppose taint tracking is L-*leaky *for a window: at most L of the queries $\mathscr{A}_{\mathsf{inject}}$ induces are presented to the gateway with $\tau = 0$. Then*

$$I\big(x; V_{\mathscr{A}_{\mathsf{inject}}}\big) \;\le\; B_\tau \log_2 M_\tau \;+\; L \log_2 M.$$

Proof. Partition the admitted queries $\mathscr{A}_{\mathsf{inject}}$ induces into those issued with $\tau = 1$ and those that escaped tainting. The first group has at most B_τ members with responses in an alphabet of size at most M_τ; the second has at most L members with responses in an alphabet of size at most M. The joint response vector therefore has entropy at most $B_\tau \log_2 M_\tau + L \log_2 M$, and data processing gives the claim. □ □

The second term can be tightened to $\min(L, B)\log_2 M$, since escaped queries also debit the untainted budget. The shape of the bound is the practical message. At our parameters the entire tainted allowance is $8\log_2 24 \approx 37$ bits, while a single escaped exact query carries up to $\log_2 |\mathscr{X}| \approx 20$ bits of recovered value: two laundered queries therefore more than double the bound, and about sixty bring it to 1,287 bits, the untainted bound of Theorem 16 itself, at which point taint gating is buying nothing. Taint coverage is thus not a binary property to be asserted in a design document but a quantity whose value enters the guarantee, and the right operational posture is to over-taint, since a false positive costs an analyst one banded answer while a false negative costs twenty bits of somebody's revenue.

We state L as a worst-case cap rather than a rate on purpose. If the number of escapes is itself random, the bound acquires an extra $H(L)$ term — $I \le B_\tau \log_2 M_\tau + H(L) + \mathbb{E}[L]\log_2 M$ — because the adversary can modulate how many queries escape. A deployment that can only measure an average escape rate should budget for the entropy of that rate as well.

6.3 Simulatability, and why prevention is unattainable

Proposition 33 (The policy is simulatable)**.** *Every rule in* Π *— set size, overlap, exposure, rowspace, budget, taint, classification — is a function of the query predicates, their induced set sizes, the history of predicates, the subject identifier and the taint bit. None of these depends on x. Hence the sequence of admit/deny decisions is simulatable from public information, and* $I(x; \text{decisions} \mid \text{pub}) = 0$.

Proof. The set sizes and the rowspace are determined by the public attributes through the characteristic vectors; the budget, subject and taint state are maintained by the gateway independently of values. A simulator with the public attributes and the query stream reproduces every decision exactly. □ □

This is a design choice with a specific pedigree. Kenthapadi et al. [28] showed that an auditor which decides on the basis of the data leaks through its refusals. We avoid that failure by construction: no rule in Π inspects a value. The price is the over-refusal quantified in §8.3 — a value-aware auditor could admit more — and we regard it as worth paying.

Proposition 34 (The decision channel cannot be closed)**.** *Suppose that at each step of a window the adversary can name two queries whose decisions it can predict and which differ — as it can whenever the policy enforces a public threshold such as a minimum query-set size. Then it transmits one bit per call through the decisions alone, and no cap on* admissions *reduces this to zero; only a cap* C_τ *on calls bounds it, at* C_τ *bits.*

Proof. To send bit b the adversary issues the query it predicts will be admitted if $b = 1$ and the one it predicts will be refused if $b = 0$; the observed decision recovers b. Denied calls consume no admission budget, so a bound on admissions does not bound the number of such probes. □ □

The hypothesis is doing real work, and we can see it fail. Against the configuration with no size rule the encoder cannot be constructed at all: no probe has a refusal the adversary can provoke, so there is no channel to measure. Against every policy that does enforce one, the same encoder delivers its message in full (§8.2). Usefulness and this channel arrive together.

Proposition 33 and Proposition 34 together give the paper's claim level precisely. The decision channel carries *nothing about x given the public attributes* — the conditioning is not decorative, since decisions do reveal query-set sizes, and in a population where those sizes correlate with the protected values an adversary who lacked the public table would learn something from them. What the channel does carry is one adversary-chosen bit per call, which suffices to exfiltrate information the model already holds: the content of the injected document itself, for example. This is containment with a stated residual, and it is why the correct headline is a bound and not a defeat.

7 Implementation

The reference gateway is a policy layer in front of an EC-ElGamal aggregation engine. Ciphertexts are pairs of secp256k1 points; SUM over a query set of size m costs $2(m-1)$ point additions, performed by the engine, never by the model host. The policy layer holds, per subject and window: a budget counter, a call counter, a per-record exposure vector, the predicate history, the taint state, and one RREF basis over $\mathrm{GF}(2^{31}-1)$ per (column, moment) audit slot. Admission is a single pass: classification, set size, taint surface, call cap, budget, overlap or exposure, then audit, so the cheap tests shed load before the expensive one runs. The set-size test deliberately precedes the taint test, so that a query which is inadmissible on its own merits is booked against the size rule whether or not the session happens to be tainted; the reverse order books 78 of the 154 correct refusals as collateral instead, inflating the apparent cost of taint gating from 112 denials to 190. We report this as an ablation rather than an assertion: both orders are implemented and measured.

The tool catalogue exposes verbs, not a query language. A generic `query(sql)` endpoint turns the gateway into a database proxy and voids every result in this paper, because the analysis assumes the gateway can compute C_φ and reason about it. Operations that cross the decryption boundary — ratios, thresholds, bucket disclosure — are separate, explicitly named tools, so that boundary crossings are individually auditable rather than implicit in a general-purpose call.

Appendix B gives the auditor in full and Appendix C the parameters. The harness, population generator, attack suite, experiment driver and figure scripts are 2,037 lines of Python, or 1,632 excluding blanks and comments. Every number in §8 except the cryptographic costs comes from a single seeded run; the cryptographic costs are wall-clock timings from a second script and are reported as medians over repeated trials.

8 Evaluation

8.1 Setup

Population. A synthetic Korean B2B corporate ledger of $n = 5{,}000$ firms with four public attributes (17 regions, 21 industry sections, 5 size bands, listed/unlisted) and protected columns for revenue, margin and headcount, plus one column deliberately omitted from the policy catalogue. Region and industry follow skewed weights and revenue is log-normal with location driven by size band. The resulting cross-classification has 1,174 occupied cells, of which 909 (77.4%) hold fewer than $k = 5$ records, and 491 records (9.8%) are uniquely identified by the four public attributes. Synthetic data is a limitation we address in §9; the low-cardinality structure it reproduces is the point, and it is characteristic of corporate rather than personal registries.

Policies. Ten configurations, from no control to the recommended stack (Table 1). Unless stated, $k = 5$, $B = 64$ admitted queries per subject per window, $B_\tau = 8$ admitted queries per tainted span, $C_\tau = 32$ tainted calls in P7, and banded responses use 24 log-spaced bands.

Attacks. Eight, specified in full in Appendix A, each run against a fresh gateway, each issued both as $\mathscr{A}_{\mathsf{inject}}$ (tainted) and as $\mathscr{A}_{\mathsf{user}}$ (authorised, untainted) so that the reader can see which defences are inert against an insider. The adversary is adaptive where adaptivity helps: A1 and A3 attempt the direct query first and build a tracker only when refused, and every tracker rotates its padding set rather than reusing one, because a weaker adversary flatters the defences in ways we found materially misleading during development. Two of the eight measure channels rather than records: A7 transmits a balanced 64-bit

Table 1: Policy configurations. $k = 5$, $B = 64$, $B_\tau = 8$, $C_\tau = 32$, 24 bands.

	Policy	Rules
P0	none	no admission control, exact responses
P1	size only	D_1
P2	+ audit (classical)	D_1, rowspace audit, weight-one test
P2b	+ audit (cell-aware)	D_1, rowspace audit, weight in $[1, k-1]$
P3	+ exposure cap	D_1, per-record exposure $E = 2$
P4	+ budget (credential)	D_1, B keyed to agent credential
P5	+ budget (subject)	D_1, B keyed to end-user subject
P6	+ taint gating	P5 + D_4
P7	**recommended**	D_1, cell-aware audit, D_3 subject, D_4 (with C_τ), D_5; exact responses when untainted
P8	banded everywhere	P6 + D_5, banded responses always

Table 2: Value-recovery success rate, as $\mathscr{A}_{\mathsf{inject}}$ (tainted) / $\mathscr{A}_{\mathsf{user}}$ (authorised). A1: two-query individual tracker. A2: nested-conjunction differencing. A3: general tracker against sub-k cells. A4: credential rotation. A5: unclassified column. A6: the same tracker routed through AVG. A8: recovery followed by export through an external-effect tool. A7 is measured in bits and appears in Table 3.

	Policy	A1	A2	A3	A4	A5	A6	A8
P0	none	1.00/1.00	1.00/1.00	1.00/1.00	1.00/1.00	1.00/1.00	1.00/1.00	1.00/1.00
P1	size only	1.00/1.00	1.00/1.00	1.00/1.00	1.00/1.00	1.00/1.00	1.00/1.00	1.00/1.00
P2	+ audit (classical)	0.00/0.00	0.00/0.00	**0.77/0.77**	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00
P2b	+ audit (cell-aware)	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00
P3	+ exposure cap	0.09/0.09	**0.71/0.71**	0.00/0.00	0.09/0.09	0.09/0.09	0.09/0.09	0.09/0.09
P4	+ budget (cred.)	0.32/0.32	0.18/0.18	0.15/0.15	**1.00/1.00**	0.32/0.32	0.32/0.32	0.32/0.32
P5	+ budget (subject)	0.32/0.32	0.18/0.18	0.15/0.15	0.32/0.32	0.32/0.32	0.32/0.32	0.32/0.32
P6	+ taint gating	0.00/**0.32**	0.00/**0.18**	0.00/**0.15**	0.00/**0.32**	0.00/**0.32**	0.00/**0.32**	0.00/**0.32**
P7	**recommended**	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00
P8	banded everywhere	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00

message through admit/deny decisions and reports the bits recovered above the constant-guessing floor, and A8 counts a value as leaked only when it is both recovered and pushed through an external-effect tool.

8.2 Attack success

Tables 2 and 3 contain eight findings worth stating separately.

Size control alone is not a control. Every value-recovery attack succeeds completely under P1, at 2.0 admitted queries per recovered record for the individual tracker and 4.0 per recovered cell for the general tracker as we implement it. This reproduces Denning et al. [16] in the agent setting and is the empirical content of Propositions 4 and 5.

The classical audit criterion has a blind spot. P2 stops every individual-compromise attack and leaves the general tracker at 0.77. The complement is exact: of the 105 sub-k cells we target, 24 are singletons, for which the weight-one test does fire, and $105 - 24 = 81$ are not, giving $81/105 = 0.77$. The cell-aware test of §5.2 closes the gap (P2b), with the caveats of Remark 10, and on this workload it costs nothing: P2 and P2b admit the same 572 of 726 benign queries.

Auditing the wrong set of aggregates is the same as not auditing. A6 is A1 with AVG substituted for SUM. Against the version of our gateway that audited only additive statistics it recovered 100 of 100 records

Table 3: Channels, in bits, as $\mathscr{A}_{\mathsf{inject}}$. A7 transmits a balanced 64-bit message through admit/deny decisions; we report bits *above* the 32-bit floor a constant-guessing decoder reaches, so 32 means the whole message arrived and 0 means no channel. Against P0 the encoder cannot be built at all. A8 is the volume that crossed the external-effect tool.

	P0	P1	P2/P2b	P3	P4/P5	P6/P8	P7
A7, decision channel (of 32)	**n/a**	32	32	2	32	8	8
A8, bits crossing E_2	2010	2010	0	181	643	0	**0**

under P2b and, as $\mathscr{A}_{\mathsf{user}}$, 32 of 100 under P7 — in the latter case limited only by the budget, with the auditor contributing nothing. With the audit slots of §5.2 in place it recovers none. We include the attack because the exemption that produced it is the natural reading of Theorem 7.

The cheap heuristic fails in a specific way. The exposure cap bites only on attacks that reuse a base predicate. Our A2 does, and still survives at 0.71; against a fully disjoint family Proposition 12 gives 1.00. We report this as a negative result because it is the defence a performance-conscious implementer will reach for first.

Budget scope is the whole ballgame. P4 and P5 differ in one line of code and by a factor of three on the rotation attack (1.00 versus 0.32). This is Theorem 13 measured, and §8.6 pushes it until the bound actually breaks.

Taint gating is inert against an insider. The P6 row is the honest one: 0.00 against $\mathscr{A}_{\mathsf{inject}}$ and unchanged against $\mathscr{A}_{\mathsf{user}}$. Taint gating defends against *steering*, not against authority. A deployment relying on it alone has protected itself against injected documents and not at all against the analyst who wanted the data anyway. This is why P7 combines it with auditing. We also note what produces P6's zeros: every probe is refused either by the size rule or by the reduced tool surface, with the direct probes of A1, A3 and A8 caught by the former and their tracker legs by the latter.

The decision channel is never closed, except by uselessness. Table 3 reports A7, which encodes a balanced 64-bit message in its choice of probe and decodes it from the decisions. We report bits above the 32-bit floor that constant guessing achieves, because raw decode accuracy flatters every partial channel. Every policy that enforces a size rule and nothing else passes the whole message: 32 of 32. The exposure cap reduces it to 2 and the taint-gated policies to 8 — the latter exactly B_τ, since once the tainted budget is spent every probe is refused regardless of its encoding. P0 has no entry: with no size rule there is no probe whose refusal the adversary can provoke, so the encoder cannot be built at all, which is the degenerate case Proposition 34 excludes. By Proposition 33 none of these bits concern x given the public metadata; the channel carries what the model already holds, not what the gateway protects.

Closing E_2 is what stops data leaving. A8 counts only values that both were recovered and crossed the external-effect `export` tool. Under P0 and P1, 2,010 bits leave. Under P2 and P2b nothing leaves because nothing was recovered; under P6 and P7 nothing leaves because the export was refused while tainted. These are different mechanisms reaching the same number, and a design should not rely on only one of them.

8.3 Benign workload

Table 4 separates denials the policy *should* make — the query set really is below k, and 154 of 726 benign analyst queries fall in that category purely because of the population's cardinality — from collateral denials, which are the real cost of the added machinery.

Three rows deserve attention. Cell-aware auditing is free on this workload: P2b admits exactly the same 572 queries as the classical auditor and the same as the bare size rule, so the sub-k protection of §5.2 costs nothing in denials and only latency. The recommended stack costs 112 collateral denials, all of them

Table 4: Benign analyst workload: 60 tasks, 726 queries, 12 subjects, 30% of sessions tainted. *Correct* denials are those where the query set is genuinely below k; *collateral* denials are everything else. Utility counts a task as served only if at least 80% of its queries return a value within 5% of the truth, so banding is charged for its imprecision.

	Policy	**adm.**	**corr.**	**coll.**	**compl.**	**util.**	**p50 (ms)**	**p95 (ms)**
P0	none	726	0	0	1.00	1.00	0.020	0.037
P1	size only	572	154	0	0.78	0.78	0.019	0.033
P2	+ audit (classical)	572	154	0	0.78	0.78	0.311	1.397
P2b	+ audit (cell-aware)	572	154	0	0.78	0.78	0.317	1.383
P3	+ exposure cap	422	154	150	0.57	0.57	0.018	0.035
P4	+ budget (cred.)	572	154	0	0.78	0.78	0.020	0.034
P5	+ budget (subject)	546	154	26	0.75	0.75	0.019	0.035
P6	+ taint gating	460	154	112	0.63	0.63	0.017	0.033
P7	**recommended**	460	154	112	0.63	**0.63**	0.216	1.288
P8	banded everywhere	460	154	112	0.63	**0.00**	0.020	0.039

attributable to the reduced tool surface during the 30% of sessions we mark tainted; that is the price of D_4, and it is the largest single utility cost in the ladder.

And P8, which bands every response, scores 0.63 on admission and 0.00 on utility: a mean relative error of 0.225 means every answer is returned and none is usable for the numeric comparisons the analyst asked for. A metric counting admission alone would have rated P8 identical to P6. We flag this because *admission rate is the wrong utility metric for a gateway that shapes responses*, and a defence evaluated that way will look free when it is not.

8.4 Cost

Figure 2 measures Proposition 6 directly. Over 1,500 queries in one subject window, size checking stays at 0.017 ms median and exposure at 0.015 ms, both flat. Hard pairwise overlap checking grows 28.7×, because it compares each new query against the whole history — the cost Chin and Özsoyoğlu [11] reported in 1982, at 1982 query rates. Rowspace auditing runs at 0.338 ms median and 10.4 ms at p95 in its classical form, and at 0.446 ms median and 7.9 ms at p95 in the cell-aware form; the cell-aware variant has the higher median and the lower tail, because refusing more queries early keeps the rank of the admitted matrix smaller later. Both grow by a factor of about 3.8 across the session.

Figure 3 puts this in context, and the context depends on an implementation choice we should not paper over. A secp256k1 point addition costs 3.40 μs, so a naive pairwise SUM over a set of size m, at $2(m-1)$ additions for the two group elements, costs 0.30 ms at the median benign query set (45 records) and 3.21 ms at the p95 set (473). A batched multi-point combine through libsecp256k1, measured at those same two sizes, does the work in 0.027 ms and 0.222 ms. Against the cell-aware auditor's 0.446 ms median and 7.9 ms p95, admission control costs between 1.5× and 17× the cryptography at the median, and between 2.5× and 35× at the tail, depending on which aggregation path the engine takes.

The engineering conclusion does not depend on which aggregation path a deployment takes: *admission control is a first-order cost, not a rounding error on the homomorphic arithmetic*, and it is the audit state, not the elliptic-curve arithmetic, that should receive the optimisation budget. Both remain small against a model round-trip, so this is a matter of allocation rather than alarm — but a deployment with a real-time conversational constraint should size itself on the auditor's p99, which is 13 ms, not on its median.

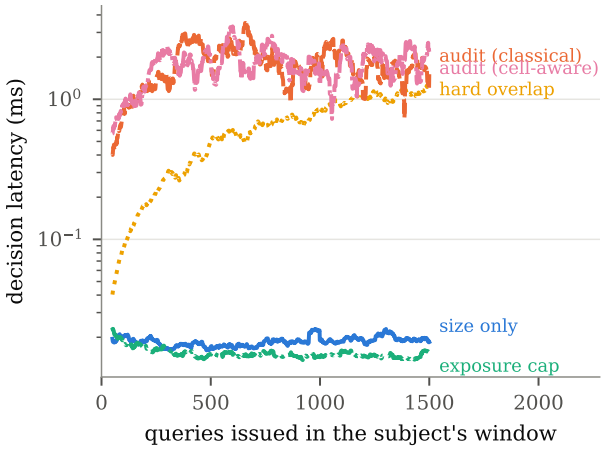


Figure 2: Per-decision latency against session length, 50-query moving mean, log scale. Size and exposure checks are flat; overlap checking and rowspace auditing are not.

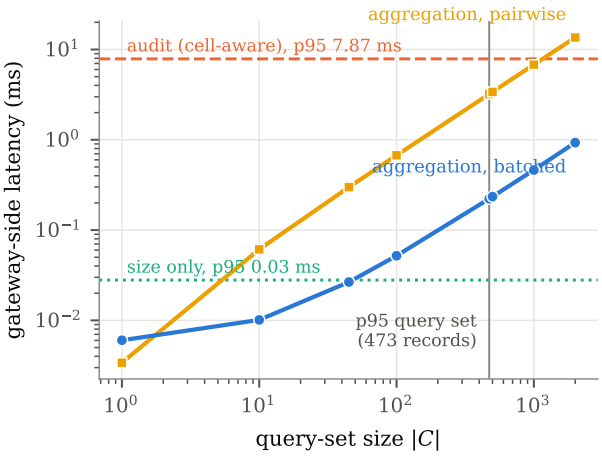


Figure 3: Admission control against the homomorphic work it guards, for both a pairwise and a batched aggregation path.

8.5 The security/utility frontier

Figure 4 sweeps $k \in \{0,3,5,10,25\}$, the three audit modes, the exposure cap $E \in \{\text{none},1,3,12\}$ and the response shaping: 120 configurations, each evaluated against the full attack suite and a 40-task benign workload. The cell-aware audit weight tracks the swept k at $w = \max(k-1,1)$, as §5.2 defines it, rather than being pinned to the default; at $k \leq 1$ there is no sub-k population and it therefore coincides with the classical test. Every configuration has deny-by-default classification and a subject-scoped budget of 64 enabled, so the sweep varies D_1 and D_2 against a fixed D_3 and D_5; the truly unprotected gateway is P0 in Table 4, not the top-left of this figure.

The result we want to state without decoration is that the frontier does not reach the corner. The highest benign utility among configurations with zero attack success is 0.75, against 1.00 for the least-restricted configuration. Containment costs twenty-five points of analyst utility in this population, and no amount of parameter tuning in the space we swept recovers them. Configurations that band every response do reach zero attack success at zero utility, which is not a trade-off but a shutdown.

The shape of the frontier in k is worth reading off, because it settles a question §5.1 raised. Holding the cell-aware auditor fixed and sweeping k gives $(k, \text{utility}, \text{ASR}) = (0,0.98,0.82)$, $(3,0.82,0.55)$, $(5,0.75,0.00)$, $(10,0.55,0.00)$, $(25,0.38,0.00)$. The knee is exactly at $k = 5$: below it the auditor alone does not close the attacks, above it the extra refusals buy nothing. That the recommended stack sits on the knee is a result of the sweep, not an assumption of it. The exposure cap at $E = 1$ also reaches zero attack success, at a utility of 0.30, confirming that the cheap heuristic can be made safe only by making it useless.

We also note what the frontier does *not* say. It compares defences at fixed population cardinality; a deployment over millions of individuals rather than thousands of firms would have far fewer sub-k cells and a much cheaper frontier. The twenty-five-point figure is a property of low-cardinality corporate data, and we would expect it to be a worst case rather than a typical one.

8.6 Bound validation

Figure 5 checks the record-recovery bound of Corollary 23 against measured leakage; by Remark 25 this, rather than the general bound of Theorem 16, is the quantity an exact-mode instantiation with $M = |\mathscr{X}|$

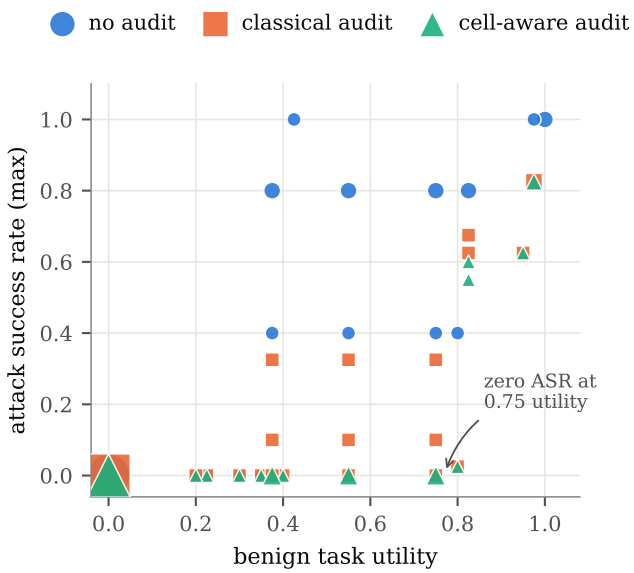


Figure 4: One hundred and twenty configurations sweeping k, audit mode, exposure cap and response shaping. Marker area is the number of configurations at that point. No configuration reaches zero attack success at full utility; the best containment costs 25 points.

actually constrains. Because the bound is stated per subject per window, we compare it against the strongest *single* attack rather than a sum across attacks, each of which runs against its own fresh gateway; summing would double-count the records that A1, A4, A5 and A6 all target.

The egress series plots the strongest channel measured against each policy, which is the export channel where values were recovered and the decision channel otherwise. Three things are visible. Policies without a budget — everything up to and including the exposure cap — have no bound at all, and we plot them as such rather than substituting a finite B after the fact. Where a bound exists and the budget is subject-scoped it holds and is loose: 1,287 bits permitted against 643 extracted under P5, and zero extracted under the recommended policy. And under P4 the value-leakage measurement exceeds the bound, 2,010 bits against 1,287: the credential-scoped budget does not bound anything, which is Theorem 13 rather than a failure of Theorem 16. Pushed to 400 targets the gap widens to 8,041 bits against the same 1,287, while the subject-scoped policy holds at 643.

We report the looseness rather than tuning it away. An information-theoretic bound that holds against an adversary we did not think of is worth more than a tight bound against the eight we did.

8.7 Compliance

Assumption 2 sets $\rho = 1$ for the bounds. Figure 6 relaxes it as a per-call probability: the model follows each injected instruction with probability ρ, and a call it declines to make is one the gateway never sees. A two-query tracker then succeeds only when both legs are issued, so recovery under P1 tracks ρ^2 — measured $2, 7, 26, 61, 100$ records against a predicted $1, 6, 25, 56, 100$ — while under the recommended policy it stays at zero throughout.

This is the concrete sense in which operating on the tool-call stream rather than on language makes the result model-independent. Compliance changes how much an unprotected gateway leaks; it does not change what a bounded one permits, because the bound is a statement about admitted responses and not about persuasion.

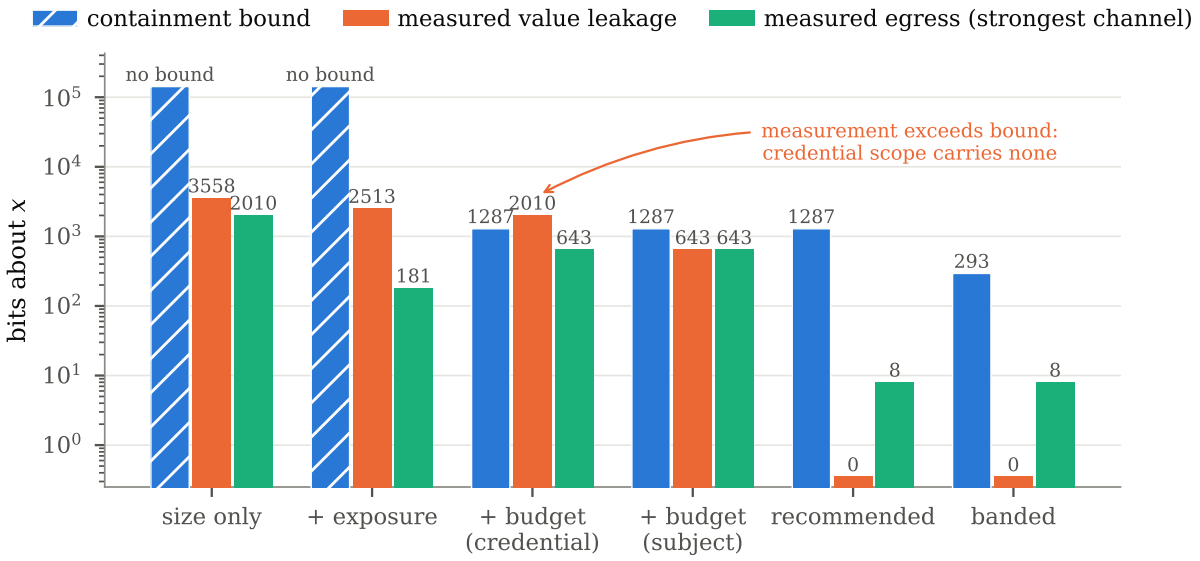


Figure 5: The record-recovery bound of Corollary 23 against measurement, log scale. Policies without a budget have no bound (hatched, plotted at the top of the axis). The credential-scoped budget is the one whose bound the measurement exceeds, which is Theorem 13 in action.

9 Discussion

What breaks this. Assumption 15 is the weakest link. Taint is supplied by the host, and laundering paths are plentiful: a summarisation step that produces "clean" text from a poisoned document, a sub-agent that returns a conclusion without provenance, a cached retrieval whose origin has been forgotten, a tool description from a third-party MCP server [3] that is read before any taint is assigned. Theorem 30 degrades gracefully in one sense — the untainted path is still covered by Theorem 16 with the larger B and M — and catastrophically in another, in that the 37-bit figure we would quote to a regulator becomes the 1,287-bit figure. A deployment should treat taint coverage as the audited control it is, and should prefer conservative tainting. The same applies to the closure of E_2: we implement and measure it against our own export tool, but a deployment's inventory of externally effectful tools is exactly as complete as its catalogue, which is the same completeness problem D_5 has.

Where the exact audit is wrong. Theorem 7 is stated over $\mathbb{Q}$. Narrow value domains, non-negativity, and public bounds on totals all give an attacker constraints the criterion does not model, and integer-programming attacks against auditing are well known in the classical literature. Our auditor is therefore necessary and not sufficient, and we would not claim otherwise in a certification context. Two further gaps are ours rather than the criterion's: the sub-k test is a relaxation of an NP-hard problem (Remark 10), and the shipped auditor works over a prime field rather than $\mathbb{Q}$ (Remark 11).

Coverage is the recurring failure mode. Three of the defects we found in our own design during this work share a shape. The auditor covered SUM and not AVG; the size rule covered individual records and not sub-k cells; the catalogue covers the columns someone remembered to classify. In each case the mechanism was correct and its domain was too small, and in each case the resulting hole was total rather than partial. We would generalise cautiously: when the caller enumerates the interface exhaustively and for free, a control that covers most of its surface provides close to none of its protection. That asymmetry does not

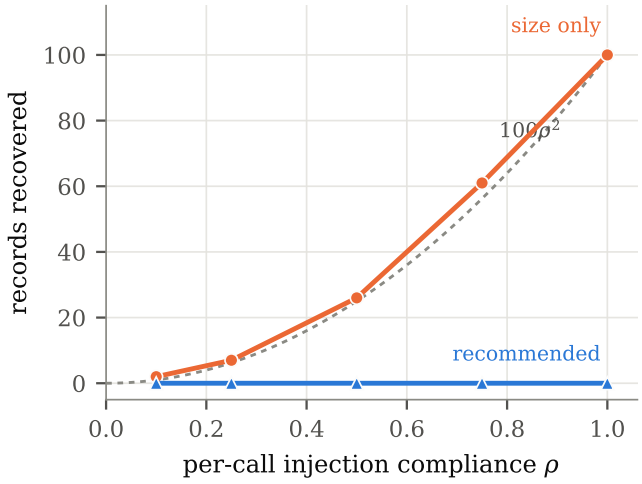


Figure 6: Records recovered against per-call injection compliance ρ. A tracker needs both of its legs, so recovery under size control alone tracks ρ^2; under the recommended policy it is flat at zero.

exist when the caller is a human analyst who probes the obvious path and stops.

Cost asymmetry cuts the other way too. Proposition 6 is about the attacker's economics, but §8.4 shows the defender's are no better: the only defence in our suite that closes the general tracker is also the only one whose cost grows with session length. Approximate or windowed auditing, rank-capped bases, and per-subject state expiry are the obvious directions, and each of them trades a measurable amount of the guarantee for throughput. We consider that trade the most important open engineering question this work raises.

Synthetic data. Every number here comes from a generated population. This is unavoidable for the attack experiments — we cannot run tracker attacks against a customer's ledger — but it means the cardinality structure, which drives the twenty-five-point utility gap, is a modelling choice rather than a measurement. The generator and seed are fixed and the population statistics are reported in full so the sensitivity can be checked.

No language models were harmed. We evaluate the admission layer, not a model. Assumption 2 makes this a strength for the bounds and a limitation for the attack results: a real end-to-end evaluation against a specific model and a real payload corpus, in the style of Debenedetti et al. [13], would test whether the tool-call streams we construct by hand are the ones an injected model actually produces. We expect ours to be stronger, since we grant $\rho = 1$ and hand-build optimal trackers, but we have not shown it, and a model that finds an attack we did not encode would falsify the attack results while leaving the bounds untouched. That asymmetry is deliberate: the bounds are the claim, and the attack suite is evidence that they are not vacuous.

What this changes for deployment. Four things, in order of how expensive they are to retrofit. Identity propagation must be in the core, not the protocol adapter, and must be present before the first tool ships (Corollary 14). A budget must exist at all, and be keyed to the subject, or there is no bound to state (Remark 17). The tool catalogue must be a whitelist, with classification automated at provisioning, or the bound does not cover the database. And the decision to sell an exact-aggregate mode is a security decision,

not a product one, because Remark 19 says the bound is weak in that mode — which is defensible, but only if it is stated.

10 Conclusion

An MCP gateway over encrypted data is a statistical database whose querier is generated, steerable and cheap. The controls the statistical-database community built for human analysts transfer in form and fail in force: minimum query-set size leaves every attack in our suite at a 1.00 success rate, the classical audit criterion misses the sub-k cell disclosures the general tracker actually achieves, and a query budget keyed to anything other than the authenticated end user admits no bound at all.

What we can offer instead is a bound. With subject-scoped budgets, cell-aware rowspace auditing over every value-bearing aggregate, a taint-gated tool surface whose external-tool closure is measured rather than assumed, and a whitelist catalogue, leakage attributable to injected content is at most $B_\tau \log_2 M_\tau$ bits per subject per window — thirty-seven at our parameters, and zero in fact, since the tainted surface returns only set sizes that were public already — and the admit/deny sequence, which cannot be closed while the policy stays useful, carries nothing about the protected column given the public metadata. The price is twenty-five points of analyst utility and an audit costing between 1.5 and 35 times the homomorphic arithmetic it guards, depending on the aggregation path.

We think a stated price and a stated residual are worth more here than a stronger claim. Three of the defects reported above were in our own design and were found by attacking it rather than by reasoning about it; the containment bounds survived all three, which is the argument for stating a bound and a falsification condition rather than a defeat.

Availability

The harness, population generator, attack suite, policy implementation and figure scripts are released as a single seeded artifact, including the AVG-exemption ablation of §5.2. The attack suite is the part we would most like reused: a defence of this kind is only as good as the streams it has been made to refuse.

References

[1] Nabil R. Adam and John C. Worthmann. Security-control methods for statistical databases: A comparative study. *ACM Computing Surveys*, 21(4):515–556, 1989. doi: 10.1145/76894.76895.

[2] Leland L. Beck. A security mechanism for statistical databases. *ACM Transactions on Database Systems*, 5(3):316–338, 1980. doi: 10.1145/320613.320617.

[3] Luca Beurer-Kellner and Marc Fischer. MCP security notification: Tool poisoning attacks, 2025. Invariant Labs, 1 April 2025. https://invariantlabs.ai/blog/mcp-security-notification-tool-poisoning-attacks.

[4] Luca Beurer-Kellner, Beat Buesser, Ana-Maria Creţu, Edoardo Debenedetti, Daniel Dobos, Daniel Fabian, Marc Fischer, David Froelicher, Kathrin Grosse, Daniel Naeff, Ezinwanne Ozoani, Andrew Paverd, Florian Tramèr, and Václav Volhejn. Design patterns for securing LLM agents against prompt injections, 2025. arXiv:2506.08837.

[5] Dan Boneh, Eu-Jin Goh, and Kobbi Nissim. Evaluating 2-dnf formulas on ciphertexts. In *Theory of Cryptography (TCC)*, volume 3378 of *LNCS*, pages 325–341. Springer, 2005. doi: 10.1007/978-3-540-30576-7_18.

[6] David Cash, Paul Grubbs, Jason Perry, and Thomas Ristenpart. Leakage-abuse attacks against searchable encryption. In *Proc. 22nd ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 668–679, 2015. doi: 10.1145/2810103.2813700.

[7] Melissa Chase and Seny Kamara. Structured encryption and controlled disclosure. In *Advances in Cryptology — ASIACRYPT 2010*, volume 6477 of *LNCS*, pages 577–594. Springer, 2010. doi: 10.1007/978-3-642-17373-8_33.

[8] Sizhe Chen, Julien Piet, Chawin Sitawarin, and David Wagner. StruQ: Defending against prompt injection with structured queries. In *Proc. 34th USENIX Security Symposium*, 2025. arXiv:2402.06363.

[9] Sizhe Chen, Arman Zharmagambetov, Saeed Mahloujifar, Kamalika Chaudhuri, David Wagner, and Chuan Guo. SecAlign: Defending against prompt injection with preference optimization. In *Proc. 2025 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2025. arXiv:2410.05451.

[10] Francis Y. L. Chin and Gultekin Özsoyoğlu. Statistical database design. *ACM Transactions on Database Systems*, 6(1):113–139, 1981. doi: 10.1145/319540.319558.

[11] Francis Y. L. Chin and Gultekin Özsoyoğlu. Auditing and inference control in statistical databases. *IEEE Transactions on Software Engineering*, SE-8(6):574–582, 1982. doi: 10.1109/TSE.1982.236161.

[12] Manuel Costa, Boris Köpf, Aashish Kolluri, Andrew Paverd, Mark Russinovich, Ahmed Salem, Shruti Tople, Lukas Wutschitz, and Santiago Zanella-Béguelin. Securing AI agents with information-flow control, 2025. arXiv:2505.23643.

[13] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents. In *Advances in Neural Information Processing Systems 37, Datasets and Benchmarks Track*, 2024. arXiv:2406.13352.

[14] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. Defeating prompt injections by design, 2025. arXiv:2503.18813.

[15] Dorothy E. Denning and Jan Schlörer. A fast procedure for finding a tracker in a statistical database. *ACM Transactions on Database Systems*, 5(1):88–102, 1980. doi: 10.1145/320128.320138.

[16] Dorothy E. Denning, Peter J. Denning, and Mayer D. Schwartz. The tracker: A threat to statistical database security. *ACM Transactions on Database Systems*, 4(1):76–96, 1979. doi: 10.1145/320064.320069.

[17] Dorothy E. R. Denning. *Cryptography and Data Security*. Addison-Wesley, Reading, MA, 1982. ISBN 0-201-10150-5.

[18] Santiago Díaz, Christoph Kern, and Kara Olive. Google's approach for secure AI agents: An introduction, 2025. Google Research.

[19] Irit Dinur and Kobbi Nissim. Revealing information while preserving privacy. In *Proc. 22nd ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems (PODS)*, pages 202–210, 2003. doi: 10.1145/773153.773173.

[20] David Dobkin, Anita K. Jones, and Richard J. Lipton. Secure databases: Protection against user influence. *ACM Transactions on Database Systems*, 4(1):97–106, 1979. doi: 10.1145/320064.320068.

[21] Cynthia Dwork. Differential privacy. In *Automata, Languages and Programming (ICALP), Part II*, volume 4052 of *LNCS*, pages 1–12. Springer, 2006. doi: 10.1007/11787006_1.

[22] Cynthia Dwork, Frank McSherry, Kobbi Nissim, and Adam Smith. Calibrating noise to sensitivity in private data analysis. In *Theory of Cryptography (TCC)*, volume 3876 of *LNCS*, pages 265–284. Springer, 2006. doi: 10.1007/11681878_14.

[23] Taher ElGamal. A public key cryptosystem and a signature scheme based on discrete logarithms. *IEEE Transactions on Information Theory*, 31(4):469–472, 1985. doi: 10.1109/TIT.1985.1057074.

[24] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world llm-integrated applications with indirect prompt injection. In *Proc. 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, pages 79–90, 2023. doi: 10.1145/3605764.3623985.

[25] Norm Hardy. The confused deputy: (or why capabilities might have been invented). *ACM SIGOPS Operating Systems Review*, 22(4):36–38, 1988. doi: 10.1145/54289.871709.

[26] Xinyi Hou, Yanjie Zhao, Shenao Wang, and Haoyu Wang. Model context protocol (MCP): Landscape, security threats, and future research directions. *ACM Transactions on Software Engineering and Methodology*, 2026. doi: 10.1145/3796519. arXiv:2503.23278.

[27] Georgios Kellaris, George Kollios, Kobbi Nissim, and Adam O'Neill. Generic attacks on secure outsourced databases. In *Proc. 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 1329–1340, 2016. doi: 10.1145/2976749.2978386.

[28] Krishnaram Kenthapadi, Nina Mishra, and Kobbi Nissim. Simulatable auditing. In *Proc. 24th ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems (PODS)*, pages 118–127, 2005. doi: 10.1145/1065167.1065183.

[29] Jon Kleinberg, Christos H. Papadimitriou, and Prabhakar Raghavan. Auditing boolean attributes. *Journal of Computer and System Sciences*, 66(1):244–253, 2003. doi: 10.1016/S0022-0000(02)00036-3.

[30] Xiaogeng Liu, Zhiyuan Yu, Yizhe Zhang, Ning Zhang, and Chaowei Xiao. Automatic and universal prompt injection attacks against large language models, 2024. arXiv:2403.04957.

[31] Ashwin Machanavajjhala, Daniel Kifer, Johannes Gehrke, and Muthuramakrishnan Venkitasubramaniam. l-diversity: Privacy beyond k-anonymity. *ACM Transactions on Knowledge Discovery from Data*, 1(1):Article 3, 2007. doi: 10.1145/1217299.1217302.

[32] Meta AI. Agents rule of two: A practical approach to AI agent security, 2025. 31 October 2025. https://ai.meta.com/blog/practical-ai-agent-security/.

[33] Model Context Protocol. Model context protocol specification, revision 2026-07-28, 2026. Agentic AI Foundation, Linux Foundation. https://modelcontextprotocol.io/specification/latest.

[34] Shubha U. Nabar, Bhaskara Marthi, Krishnaram Kenthapadi, Nina Mishra, and Rajeev Motwani. Towards robustness in query auditing. In *Proc. 32nd International Conference on Very Large Data Bases (VLDB)*, pages 151–162, 2006.

[35] National Institute of Standards and Technology. Artificial intelligence risk management framework (ai rmf 1.0). Technical Report NIST AI 100-1, NIST, 2023.

[36] Muhammad Naveed, Seny Kamara, and Charles V. Wright. Inference attacks on property-preserving encrypted databases. In *Proc. 22nd ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 644–655, 2015. doi: 10.1145/2810103.2813651.

[37] OWASP GenAI Security Project. OWASP top 10 for LLM applications 2025, 2025. https://genai.owasp.org/llm-top-10/.

[38] Fábio Perez and Ian Ribeiro. Ignore previous prompt: Attack techniques for language models, 2022. arXiv:2211.09527. ML Safety Workshop, NeurIPS 2022.

[39] Julien Piet, Maha Alrashed, Chawin Sitawarin, Sizhe Chen, Zeming Wei, Elizabeth Sun, Basel Alomair, and David Wagner. Jatmo: Prompt injection defense by task-specific finetuning. In *Computer Security — ESORICS 2024*, LNCS, pages 105–124. Springer, 2024. doi: 10.1007/978-3-031-70879-4_6.

[40] Brandon Radosevich and John Halloran. MCP safety audit: LLMs with the model context protocol allow major security exploits, 2025. arXiv:2504.03767.

[41] Jan Schlörer. Disclosure from statistical databases: Quantitative aspects of trackers. *ACM Transactions on Database Systems*, 5(4):467–492, 1980. doi: 10.1145/320610.320645.

[42] Tianneng Shi, Jingxuan He, Zhun Wang, Hongwei Li, Linyu Wu, Wenbo Guo, and Dawn Song. Progent: Securing AI agents with privilege control, 2025. arXiv:2504.11703.

[43] Latanya Sweeney. k-anonymity: A model for protecting privacy. *International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems*, 10(5):557–570, 2002. doi: 10.1142/S0218488502001648.

[44] Alexander Vardy. The intractability of computing the minimum distance of a code. *IEEE Transactions on Information Theory*, 43(6):1757–1766, 1997. doi: 10.1109/18.641542.

[45] Simon Willison. The dual LLM pattern for building AI assistants that can resist prompt injection, 2023. https://simonwillison.net/2023/Apr/25/dual-llm-pattern/.

[46] Fangzhou Wu, Ethan Cecchetti, and Chaowei Xiao. System-level defense against indirect prompt injection attacks: An information flow control perspective, 2024. arXiv:2409.19091.

[47] Yuhao Wu, Franziska Roesner, Tadayoshi Kohno, Ning Zhang, and Umar Iqbal. IsolateGPT: An execution isolation architecture for LLM-based agentic systems. In *Proc. 32nd Network and Distributed System Security Symposium (NDSS)*, 2025. arXiv:2403.04960.

[48] Yixuan Yang, Cuifeng Gao, Daoyuan Wu, Yufan Chen, Yingjiu Li, and Shuai Wang. MCPSecBench: A systematic security benchmark and playground for testing model context protocols, 2025. arXiv:2508.13220.

A Attack constructions

A1, two-query individual tracker. For target t with characterising conjunction ψ_t, select a padding predicate θ from the single-attribute predicates with $k \le |C_\theta| \le n-k-1$ and $t \notin C_\theta$, rotating the choice across targets. Issue $\mathrm{SUM}(\theta \vee \{t\})$ and $\mathrm{SUM}(\theta)$; the difference is x_t. The adversary first attempts $\mathrm{SUM}(\psi_t)$ directly and falls back to the tracker only on refusal.

A2, nested-conjunction differencing. Find pairs $(\alpha, \alpha \wedge \beta)$ over public attributes with $|C_{\alpha\wedge\beta}| = 1$ and $|C_\alpha| \geq k+1$; the difference of the two SUM aggregates is the single record's value. Needs no disjunction, so it survives a conjunction-only filter API.

A3, general tracker. As Proposition 5, targeting the two-attribute cells of size $1 \leq |C| < k$, with padding rotated per cell and $q(T), q(\neg T)$ re-issued per padding set so that per-record exposure does not concentrate.

A4, credential rotation. A1 with a fresh agent credential every two calls and a constant end-user subject.

A5, unclassified column. A1 against a column present in the physical schema and absent from the policy catalogue.

A6, AVG tracker. A1 with AVG substituted for SUM; the record is reconstructed as $\text{avg}(\theta \vee \psi_t)\,(m+1) - \text{avg}(\theta)\,m$, with $m = |C_\theta|$ computed from the published catalogue.

A7, decision channel. The adversary holds a 64-bit message. To send bit b it issues a query over a cell it knows to be above the published minimum size when $b = 1$ and below it when $b = 0$, then decodes the message from the admit/deny sequence. The message is balanced, and success is the number of bits decoded *above* the 32-bit floor a constant-guessing decoder reaches.

A8, egress via export. A1 followed by a call to an external-effect `export` tool carrying the recovered values. A value counts as leaked only if it is both recovered and exported, so the attack measures E_2 rather than assuming it.

B Auditor

The auditor maintains one RREF basis per audit slot, a slot being a (column, moment) pair: SUM and AVG share the first-moment slot for their column, VAR occupies the second-moment slot, and COUNT is exempt because it reveals only a set size. Each basis is kept over $\text{GF}(p)$ with $p = 2^{31} - 1$, chosen so that products fit in a signed 64-bit word. On a candidate query with characteristic vector v: reduce v against the current basis; if it reduces to zero the query is linearly dependent and adds nothing, so admit. Otherwise normalise the reduced row at its pivot, back-substitute it into the existing rows, and test the resulting row set for any row of Hamming weight in $[1, w]$, where $w = 1$ gives the classical criterion and $w = k - 1$ the cell-aware one. If the test fires, refuse and discard the candidate row; otherwise commit it. The basis is rank-capped, and reaching the cap causes subsequent independent queries to be admitted without being recorded — a deliberate availability-over-security choice that a deployment should instead resolve by expiring the window.

C Parameters

Symbol	Meaning	Value
n	records in the population	5,000
k	minimum query-set size	5
B	admitted queries per subject per window	64
B_τ	admitted queries per tainted span	8
C_τ	*calls* per tainted span	32
M	response alphabet, banded mode	24
$\lvert\mathscr{X}\rvert$	value domain, exact mode	$2^{20.1}$
E	per-record exposure cap	2 (P3); $\{1,3,12\}$ (sweep); 8 (cost)
r	hard pairwise overlap cap	200 (cost experiment)
p	auditor field modulus	$2^{31}-1$
ρ	injection compliance	1.0 unless swept
	tainted fraction of benign sessions	0.30
	benign relative-error tolerance	0.05